CUSTOMER CHURN PREDICTION SQL ANALYSIS

TABLE INSERT DATAS:

```
-- ========================================
-- TABLE 1 : Customers
-- ========================================

CREATE TABLE Customers (
    customer_id VARCHAR(20) PRIMARY KEY,
    gender VARCHAR(10),
    tenure INT,
    contract_type VARCHAR(50),
    monthly_charges FLOAT,
    churn INT
);

INSERT INTO Customers VALUES
('C101', 'Male', 12, 'Month-to-Month', 1200.50, 1),
('C102', 'Female', 24, 'One Year', 950.00, 0),
('C103', 'Male', 36, 'Two Year', 1500.75, 0),
('C104', 'Female', 6, 'Month-to-Month', 700.25, 1),
('C105', 'Male', 18, 'One Year', 1100.00, 0);


-- ========================================
-- TABLE 2 : Support_Tickets
-- ========================================

CREATE TABLE Support_Tickets (
    ticket_id VARCHAR(20) PRIMARY KEY,
    customer_id VARCHAR(20),
    issue_type VARCHAR(100),
    resolution_time FLOAT,
    satisfaction_score FLOAT,

    FOREIGN KEY (customer_id)
    REFERENCES Customers(customer_id)
);

INSERT INTO Support_Tickets VALUES
('T201', 'C101', 'Billing Issue', 5.5, 3.5),
('T202', 'C102', 'Network Problem', 2.0, 4.5),
('T203', 'C103', 'Technical Support', 8.0, 4.0),
('T204', 'C101', 'Service Downtime', 6.5, 2.5),
('T205', 'C104', 'Billing Issue', 4.0, 3.0);


-- ========================================
-- TABLE 3 : Marketing_Campaign
-- ========================================

CREATE TABLE Marketing_Campaign (
    campaign_id VARCHAR(20) PRIMARY KEY,
```

```
    customer_id VARCHAR(20),
    campaign_type VARCHAR(100),
    response VARCHAR(20),

    FOREIGN KEY (customer_id)
    REFERENCES Customers(customer_id)
);

INSERT INTO Marketing_Campaign VALUES
('M301', 'C101', 'Email Campaign', 'Responded'),
('M302', 'C102', 'SMS Campaign', 'Ignored'),
('M303', 'C103', 'Phone Call', 'Responded'),
('M304', 'C104', 'Email Campaign', 'Declined'),
('M305', 'C105', 'SMS Campaign', ‘Responded');
```

SQL QUERY :

```
select * from Customers;
select * from Marketing_Campaign;
select * from Support_Tickets;
```

##BASIC DATA EXPLORATION

##Find total number of customers

```
select count(*) from Customers;
```

##Find total churned customers

```
select * from Customers
where churn = 1;
```

##Calculate total revenue generated

```
select sum(monthly_charges) as total_rev
from Customers;
```

##Find average monthly charges

```
select Customer_id , monthly_charges,  round(avg(monthly_charges)) as avg_month
from Customers
group by Customer_id;
```

##Find unique contract types

```
select count(distinct(contract_type)) from customers;
```

##Count customers by gender

```
select gender , count(*) from Customers
group by gender;
```

##Find customers with tenure greater than 24 months

```
select * from Customers
where tenure > 24;
```

##Find top 10 highest paying customers

```
select customer_id , gender, (tenure * monthly_charges) as total_pay
from Customers
group by customer_id , gender
order by total_pay Desc;
```

##JOIN TASK

##Combine customers and support tickets tables

```
select c.customer_id , c.gender , st.issue_type , st.resolution_time , st.satisfaction_score
from Customers c
join Support_Tickets st
on c.customer_id = st.customer_id
group by c.customer_id , c.gender , st.issue_type , st.resolution_time , st.satisfaction_score ;
```

##Show all customers even if no support ticket exists

```
select c.customer_id , c.gender , st.issue_type , st.resolution_time , st.satisfaction_score
from Customers c
left join Support_Tickets st
on c.customer_id = st.customer_id
where st.customer_id IS NULL
group by c.customer_id , c.gender , st.issue_type , st.resolution_time , st.satisfaction_score;
```

##Find customers who responded to campaigns

```
select c.customer_id , c.gender  , mc.campaign_type , mc.response , mc.campaign_id
from Customers c
join Marketing_Campaign mc
on c.customer_id = mc.customer_id
where mc.customer_id IS NOT NULL
group by c.customer_id , c.gender  , mc.campaign_type , mc.response , mc.campaign_id;
```

##Find customers with both support tickets and churn

```
select c.customer_id , c.gender , c.churn , st.issue_type , st.resolution_time , st.satisfaction_score
from Customers c
left join Support_Tickets st
on c.customer_id = st.customer_id
where st.customer_id IS NOT NULL and c.churn = 1
group by c.customer_id , c.gender , c.churn , st.issue_type , st.resolution_time ,
st.satisfaction_score;
```

##Display customer revenue along with campaign response

```
select c.customer_id , c.gender , c.churn , (c.tenure * c.monthly_charges) as total_pay  ,
mc.campaign_id , mc.campaign_type , mc.response
from Customers c
join Marketing_Campaign mc
on c.customer_id = mc.customer_id
group by c.customer_id , c.gender , c.churn ,  mc.campaign_id , mc.campaign_type ,
mc.response
order by total_pay desc;
```

##Find average satisfaction score by contract type

```
select c.customer_id , c.gender , c.contract_type , st.issue_type , st.resolution_time ,
round(avg(st.satisfaction_score)) as avg_satisfraction_score
from Customers c
join Support_Tickets st
on c.customer_id  =  st.customer_id
group by c.customer_id , c.gender , c.contract_type , st.issue_type , st.resolution_time ,
st.satisfaction_score
order by avg_satisfraction_score Desc;
```

##AGGREGATION TASK

##Calculate churn rate by contract type

```
select contract_type , churn
from Customers
group by contract_type , churn ;
```

##Find total revenue by internet service type

```
select sum(tenure * monthly_charges) as total_pay , contract_type
from Customers
group by contract_type
order by total_pay desc;
```

##Find average resolution time per issue type

```
select round(avg(resolution_time)) as avg_resolution , issue_type
from Support_Tickets
group by issue_type
order by avg_resolution desc;
```

##Find top 5 issue categories

```
select count(distinct(issue_type)) from Support_Tickets;
```

##Calculate customer lifetime value

```
select sum(tenure * monthly_charges) as lifetime_Value , customer_id , gender
from Customers
group by customer_id , gender
order by lifetime_Value desc;
```

##Find customers with highest total charges

```
select sum(tenure * monthly_charges) as lifetime_Value , customer_id , gender
from Customers
group by customer_id , gender
order by lifetime_Value desc
limit 1;
```

##CASE STATEMENT TASK

##Classify customers as High/Medium/Low value

```
select customer_id , gender , monthly_charges ,
        case
                when monthly_charges > 1200 then 'high'
      when monthly_charges < 1000 then 'low'
      else 'medium'
        end as customer_segment
from Customers;
```

##Categorize customers based on tenure

```
select customer_id , gender , tenure,
        case
   when tenure > 20 then 'HIGH'
   when tenure <=10 then 'LOW'
   else 'MEDIUM'
END AS tenure_segmentation
FROM Customers;
```

##Create churn risk categories

```
select customer_id , gender , churn ,
        case
                when churn > 0 then 'NO CHURN'
      else "CUSTOMER CHURNED"
        end as churn_segmentation
from Customers;
```

```
##Identify premium customers
select tenure , gender , customer_id
from Customers
```

```
where tenure > 12;
```

##Label customers based on satisfaction score

```
select customer_id , ticket_id , resolution_time , issue_type , satisfaction_score,
        case
                when satisfaction_score > 3 then "CUSTOMER SATISFIED"
      else "CUSTOMER UNSATISFIED"
        end as customer_satisfraction
from Support_Tickets;
```

##SUBQUERY TASKS

##Find customers paying above average monthly charges

```
select gender , customer_id , tenure , monthly_charges
from Customers
where monthly_charges > (select avg(monthly_charges) as average_charges from Customers);
```

##Find customers with highest tenure in each contract type

```
select gender , tenure , contract_type
from Customers c
where tenure = (select max(tenure) as max_ten from Customers where contract_type =
c.contract_type);
```

##Find customers whose revenue exceeds overall average

```
select customer_id , gender , tenure , contract_type , (tenure * monthly_charges) as
total_revenvure
from Customers
where (tenure * monthly_charges) > (select round(avg(tenure * monthly_charges) , 2) as avg_rev
from Customers);
```

##Find customers with more tickets than average

```
SELECT
   c.customer_id,
   c.gender,
   c.contract_type,
   COUNT(st.ticket_id) AS total_tickets
FROM Customers c
JOIN Support_Tickets st
ON c.customer_id = st.customer_id

GROUP BY
   c.customer_id,
   c.gender,
   c.contract_type

HAVING COUNT(st.ticket_id) > (

   SELECT AVG(ticket_count)
   FROM (
      SELECT COUNT(ticket_id) AS ticket_count
      FROM Support_Tickets
      GROUP BY customer_id
   ) avg_table
);
```

##Find campaigns with above-average response rate

```
SELECT
   campaign_type,
   COUNT(
      CASE
```

```
            WHEN response = 'Responded'
            THEN 1
        END
    ) * 100.0 / COUNT(*) AS response_rate
FROM Marketing_Campaign
GROUP BY campaign_type
HAVING response_rate > (
    SELECT AVG(response_percentage)
    FROM (
        SELECT
            COUNT(
                CASE
                    WHEN response = 'Responded'
                    THEN 1
                END
            ) * 100.0 / COUNT(*) AS response_percentage
        FROM Marketing_Campaign
    ) avg_response_table
);
```

##CTE COMMANDS

##Create customer revenue summary using CTE

```
with customer_revenue as (
            select customer_id ,
    gender,
    tenure,
    contract_type,
    monthly_charges,
    churn,
    (tenure * monthly_charges) as total_rev
    from Customers
)
select customer_id ,
    gender,
    tenure,
    contract_type,
    monthly_charges,
    churn,
    total_rev
    from customer_revenue
    order by total_rev desc;
```

##Calculate churn percentage using CTE

```
with churn_percentage as (
            select count(*) as total_customers,
    sum(
                        case
                                    when churn = 1
            then 1
            else 0
                        end
        ) as churn_customers
            from Customers
)

select total_customers , churn_customers,
            round(churn_customers * 100.0 / total_customers  , 2) as churn_percent
from churn_percentage;
```

##Find high-risk customers using CTE

```
with high_risk_cust as (

    select
    customer_id,
    gender,
    tenure,
    contract_type,
    monthly_charges
    from Customers
    where churn = 1
    )

select * from high_risk_cust;
```

##Create support performance report

```
with support_report as(

        select c.customer_id,
    c.gender,
    c.tenure,
    c.contract_type,
    c.monthly_charges,
    mc.campaign_type,
    mc.campaign_id,
    mc.response,
    st.issue_type,
    st.satisfaction_score
    from Customers c
    join Marketing_Campaign mc
    on c.customer_id = mc.customer_id
    join Support_Tickets st
    on mc.customer_id = st.customer_id
    group by c.customer_id,
    c.gender,
    c.tenure,
    c.contract_type,
    c.monthly_charges,
    mc.campaign_type,
    mc.campaign_id,
    mc.response,
    st.issue_type,
    st.satisfaction_score
)

select * from support_report;
```

##Rank customers by revenue using CTE

```
with rank_customers as(
        select customer_id,
    gender,
    tenure,
    contract_type,
    monthly_charges,
    (tenure * monthly_charges) as total_rev,
    rank() over(order by (tenure * monthly_charges) desc)
    from Customers
)
select * from rank_customers;
```

##WINDOW FUNCTIONS

##Rank customers based on revenue

```
select customer_id,
    gender,
    tenure,
    contract_type,
    monthly_charges,
    (tenure * monthly_charges) as total_rev,
    rank() over(order by (tenure * monthly_charges) desc)
from Customers;
```

##Find top customer in each contract type

```
select customer_id,
    gender,
    tenure,
    contract_type,
    monthly_charges,
    (tenure * monthly_charges) as total_rev
    from Customers
    group by customer_id,
    gender,
    tenure,
    contract_type,
    monthly_charges
    order by total_rev desc
    limit 1;
```

##Calculate running revenue total

```
select
        customer_id , gender , tenure , contract_type,
    sum(monthly_charges) over ( order by monthly_charges) as runnning_Rev
from Customers;
```

##Find previous month's revenue

```
select
        customer_id , gender , tenure , contract_type,
    lag(monthly_charges) over() as lag_month
from Customers;
```

##Assign row numbers to customers

```
select
        customer_id , gender , tenure , contract_type,(tenure * monthly_charges) as total_rev ,
    row_number() over(order by (tenure * monthly_charges) desc) as row_rev
from Customers;
```

##ADVANCE BUSINESS LOGIC

##Find customers likely to churn based on support activity

```
select * from Customers
where churn = 1;
```

##Identify low-engagement high-value customers

```
select * from Customers
where tenure < 10 and monthly_charges > 1000;
```

##Find which campaigns reduce churn most

```
select * from Marketing_Campaign
```

where response = "Responded";

##Find support teams with best satisfaction scores

```
select mc.campaign_type , mc.campaign_id , mc.response , st.issue_type
from Marketing_Campaign mc
join Support_Tickets st
on mc.customer_id = st.customer_id
where st.satisfaction_score > 2
group by mc.campaign_type , mc.campaign_id , mc.response , st.issue_type;
```